

— Bir kurye programı; Admin panel, Web Uygulama Müşteri tarafı, Kurye ve Müşteri için mobil uygulama. Çok kolay kullanılabilir ve çok gelişmiş olmalı.

Bu tip bir sistemi sıradan bir "kurye uygulaması" olarak değil, **lojistik işletim sistemi (Logistics Operating System - LOS)** olarak tasarlırsak yıllarca geliştirilebilir ve farklı sektörlerde de satılabilir (yemek, e-ticaret, kargo, teknik servis, eczane, market vb.).

Ben olsam sistemi 5 ana parçaya ayırıyorum.

1. Yönetim Paneli (Admin)

Burası şirketin beyni olacak.

Dashboard

- Anlık sipariş sayısı
- Online kurye sayısı
- Bekleyen teslimatlar
- Günlük gelir
- Haftalık gelir
- Aylık gelir
- Aktif müşteriler
- Isı haritası
- Canlı harita

Sipariş Yönetimi

- Yeni Sipariş
- Bekleyen
- Kurye Atanacak
- Kurye Yolda
- Teslim Edildi
- İptal
- İade

Filtreleme

- Tarih
- Kurye
- Bölge
- Firma
- Durum

Kurye Yönetimi

Her kurye için

- Profil

- Fotoğraf
- Kimlik
- Ehliyet
- Araç Bilgisi
- Motor/Bisiklet/Otomobil
- Plaka
- Sigorta

Canlı

- Konumu
- Hızı
- Batarya
- Son Görölme
- Günlük teslimat
- Performans

Firma Yönetimi

Şirketler

Market

Restoran

Eczane

Kargo

Mağaza

Kurye firmaları

Müşteri Yönetimi

- Profil
- Adresler
- Favoriler
- Geçmiş Siparişler
- Puan
- Şikayetler

Finans

- Tahsilatlar
- Kurye Hakedişleri
- Komisyonlar
- Primler
- Cezalar
- İadeler

Kampanyalar

Kupon

İndirim

Promosyon

Referans

Sadakat

Bildirim Merkezi

SMS

Push

WhatsApp

Mail

Toplu Bildirim

Raporlar

Kurye Performansı

Gelir

Harita

Yoğunluk

Teslim Süresi

Yetkilendirme

Rol sistemi

Admin

Operator

Muhasebe

Çağrı Merkezi

Bölge Müdürü

Super Admin

2. Web Müşteri Paneli

Kurumsal müşteriler için.

Sipariş oluşturma

Adres defteri

Toplu sipariş

Excel yükleme

Takip ekranı

Canlı harita

Teslim geçmişi

Fatura

Raporlar

API anahtarları

Webhook yönetimi

3. Mobil Müşteri Uygulaması

Android

iPhone

Ana ekran

Yeni Kurye Çağır

Haritada Konum

Yakındaki Kurye

Tahmini Süre

Sipariş

Alım Adresi

Teslim Adresi

Fotoğraf

Not

Telefon

Teslimat tipi

Ödeme

Kart

Nakit

QR

Cüzdan

Canlı Takip

Harita

Kurye Nerede

ETA

Bildirimler

Geçmiş

Tekrar Sipariş

Favoriler

4. Kurye Mobil

Bu uygulama çok kritik.

Giriş

Telefon

OTP

Biometrik

Online / Offline

Tek tuş

Yeni İş

Bildirim

Reddet

Kabul Et

Navigasyon

Google Maps

Apple Maps

Yandex

Teslim Alma

QR

Barkod

Fotoğraf

İmza

Teslim

Fotoğraf

İmza

PIN

QR

Kazanç

Bugün

Hafta

Ay

İstatistik

Puan

Performans

Teslim

Mesafe

5. API

REST

GraphQL

WebSocket

Webhook

SDK

Canlı Harita

En önemli bölüm.

Gösterilecekler

Tüm kuryeler

Online

Offline

Siparişler

Yoğunluk

Isı Haritası

Gerçek Zamanlı

WebSocket

Socket.io

Redis Pub/Sub

Kafka (çok büyük sistemlerde)

Bildirimler

Firebase Push

APNs

SMS

WhatsApp

Mail

Yapay Zeka

Bence en büyük fark burada olmalı.

AI Dispatcher

En uygun kuryeyi otomatik seçer.

AI ETA

Trafik

Yağmur

Yoğunluk

Motor tipi

Teslim süresini hesaplar.

AI Route

Bir kuryeye 5–10 teslimatı en verimli sırayla planlar.

AI Fraud Detection

Sahte teslimat

Konum sahteciliği

GPS hilesi

Çok hızlı hareket

Şüpheli davranış

AI Customer Support

7/24 chatbot

AI Analytics

En yoğun bölgeler

En yoğun saatler

Yeni şube önerileri

Kurye ihtiyacı tahmini

Teknoloji

Backend

- Rust (Axum) veya Go (Fiber) – yüksek performanslı mikroservis mimarisi
- PostgreSQL
- Redis
- NATS veya Kafka (ölçeğe göre)
- MinIO / S3
- Meilisearch veya OpenSearch

Web

- React
- Next.js
- Tailwind CSS
- TypeScript

Mobil

- Flutter (tek kod tabanı ile Android + iOS) veya React Native
- Offline çalışma desteği
- Arka planda konum güncelleme
- Push bildirimleri

Haritalar

- Google Maps
- OpenStreetMap (Leaflet/MapLibre)

- Apple Maps (iOS entegrasyonu)

Bulut

- Docker
- Kubernetes (büyüme aşamasında)
- Nginx
- Prometheus
- Grafana
- Loki

Güvenlik

- JWT + Refresh Token
- OAuth2/OIDC desteği
- Çok faktörlü kimlik doğrulama
- Cihaz yönetimi
- Uçtan uca TLS
- Ayrıntılı denetim kayıtları (Audit Log)
- KVKK/GDPR uyumlu veri yönetimi
- Yedekleme ve felaket kurtarma planı

Benim Eklemek İstedığım En Büyük Özellik

Bu projeyi sadece bir kurye uygulaması olarak değil, **çok kiracılı (multi-tenant) SaaS platformu** olarak tasarladım.

Böylece aynı altyapıyla:

- Bir restoran zinciri,
- Bir market,
- Bir e-ticaret firması,
- Bir eczane,
- Bir teknik servis şirketi,
- Yerel bir kurye firması,

kendi alan adları, logoları, renkleri ve kullanıcılarıyla sistemi bağımsız olarak kullanabilir. Bu yaklaşım hem uzun vadede bakım maliyetini düşürür hem de ürünü farklı müşterilere lisanslayarak gelir elde etmeyi kolaylaştırır.

Bence bu mimariyle geliştirilecek ürün, sadece müşterinizin ihtiyacını karşılamakla kalmaz; ileride satılabilecek profesyonel bir lojistik platformuna dönüşebilir. Bu, ileride sizin de yeni projelerinizde kullanabileceğiniz güçlü ve yeniden kullanılabilir bir altyapı oluşturur.

1) Akıllı Dağıtım Motoru (Dispatch Engine)

En önemli modül.

Kurye seçerken sadece mesafeye bakmayacak.

Hesaplayacağı parametreler:

- Trafik
- Hava Durumu
- Kurye Puanı
- Araç Tipi
- Yakıt Tüketimi
- Boş Kapasite
- Paket Boyutu
- Müşteri Önceliği
- VIP Müşteri
- Teslimat Süresi (SLA)
- Bölgesel Yoğunluk

Sonra AI puanlayacak.

Courier Score

Distance

+

Traffic

+

Experience

+

Vehicle

+

Battery

+

Current Load

+

Rating

=

Best Courier

2) Harita Motoru

Google Maps tek başına yeterli değil.

Destek:

- Google Maps
- OpenStreetMap
- MapLibre
- HERE Maps
- TomTom

İstenirse müşteri seçebilecek.

3) Bölge Yönetimi (Geofencing)

Admin haritadan çizecek.

Örneğin

İstanbul

↓

Kadıköy

↓

Moda

↓

Rıhtım

↓

Sokak

Kurye sadece kendi bölgesinde görev alacak.

4) Dinamik Fiyatlandırma

Uber mantığı.

Yoğunluk artınca

Normal

100 TL

↓

Yoğun

125 TL

↓

Çok Yoğun

165 TL

Tamamen otomatik.

5) AI ETA

Her saniye güncellenecek.

Hesap:

GPS

•

Trafik

•

Yağmur

•

Kurye Hızı

•

Geçmiş Teslimatlar



Tahmini Varış

6) Akıllı Rota

Bir kuryede

15 teslimat varsa

AI optimum sırayı oluşturacak.

Yaklaşık %15–30 yakıt tasarrufu sağlayabilir.

7) Canlı İzleme

Admin

Kurye

Müşteri

aynı anda görebilecek.

Haritada:



Kurye



Paket



Teslim Noktası



Bekleyen

8) Chat

Kurye ↔ Müşteri

Kurye ↔ Firma

Kurye ↔ Admin

Destek:

- Fotoğraf
- Video

- Dosya
- Konum
- Ses

9) Dahili Arama

VoIP

WebRTC

Numara gizleme

Çağrı kayıtları

10) QR Teslim

Pakete QR

Kurye okutur.

Teslimde tekrar okutur.

Yanlış paket teslim edilemez.

11) NFC

İstenirse

NFC ile teslim.

12) Barkod

EAN

QR

Code128

DataMatrix

PDF417

13) Fotoğraflı Teslim

Kapıya bırakıldı

↓

Fotoğraf

↓

GPS

↓

Saat

↓

İmzalı kayıt

14) Elektronik İmza

Mobil ekran

↓

İmza

↓

PDF

↓

Arşiv

15) Dijital Evrak

İrsaliye

Fatura

Teslim Tutanağı

PDF

16) Kurye Cüzdanı

Kazanç

↓

Bakiye

↓

Çekim Talebi

↓

IBAN

↓

Kripto (Opsiyonel)

17) Prim Sistemi

100 teslimat

↓

500 TL Bonus

200 teslimat

↓

1000 TL

18) Ceza Sistemi

Geç Teslim

↓

Puan düşer

İptal

↓

Ceza

19) Performans Sistemi

Kurye

★★★★★

98%

Teslim Süresi

4.9

Müşteri Puanı

20) Rozet Sistemi

Gold Courier

Elite Courier

VIP Courier

Fast Courier

Perfect Courier

21) Müşteri Sadakat

Her teslimat

↓

Puan

↓

İndirim

↓

Kupon

22) Referans Sistemi

Arkadaşını getir



İkisine de puan

23) Şikayet Yönetimi

Ticket



Fotoğraf



Video



Çözüm



Kapanış

24) SLA Yönetimi

Örneğin

Market

30 dk

Restaurant

45 dk

Kargo

24 Saat

Geçilirse uyarı.

25) Akıllı Dashboard

Canlı KPI'lar:

- Ortalama teslim süresi
- Başarılı teslimat oranı
- İptal oranı
- Kurye doluluk oranı
- Bölgesel yoğunluk
- Saatlik sipariş grafiği
- Günlük gelir
- Aktif kullanıcılar
- Harita ısı analizi

26) İş Akışı Tasarımcısı (Workflow Builder)

Kod yazmadan sürükle-bırak mantığıyla kurallar oluşturulabilir.

Örnek:

Sipariş Geldi

↓

Ödeme Alındı

↓

Kurye Ata

↓

Kurye Kabul

↓

Teslim Al

↓

Teslim Et

↓

SMS Gönder

↓

Fatura Kes

27) Otomasyon Motoru

Örnek kurallar:

Sipariş > 500 TL

↓

Yönetici Onayı
veya

Kurye
15 dakika hareket etmezse

↓

Alarm

28) API Marketplace

REST

GraphQL

Webhook

SDK

Zapier

n8n

Make

ERP Entegrasyonları

Muhasebe

Pazaryeri

CRM

29) AI Destek

Gerçek üretken yapay zekâ özellikleri:

- Operasyon asistanı (yöneticilerin doğal dille rapor alması)
- Müşteri destek botu
- Kurye performans önerileri
- Talep tahmini
- Yoğunluk analizi
- Dinamik fiyat önerileri
- Riskli sipariş tespiti
- Anomali ve sahtecilik analizi

30) İşletim Merkezi (Command Center)

Bu ekran, büyük lojistik firmalarının kullandığı operasyon merkezine benzer.

Tek ekranda:

- Tüm kuryeler
- Tüm siparişler
- Tüm bölgeler
- Canlı harita
- Hava durumu
- Trafik durumu
- Kritik alarmlar
- SLA ihlalleri
- Sistem sağlığı
- Sunucu durumu
- API durumu
- WebSocket bağlantıları
- Canlı loglar

Gelecek Vizyonu (V3)

Bu altyapıyı modüler tasarlırsak aynı çekirdek sistemi kullanarak şu ürünleri de geliştirebiliriz:

-  Taksi çağırma
-  Nakliye yönetimi
-  Market teslimatı
-  Yemek teslimatı
-  Eczane teslimatı
-  Teknik servis yönetimi
-  Evde sağlık hizmetleri
-  Kargo ve dağıtım
-  Filo yönetimi
-  Saha operasyonları

-  Saha personeli görev yönetimi

Yani tek bir platformdan farklı sektörlere uyarlanabilen, lisanslanabilir bir ürün ailesi ortaya çıkar.

Benim önerim

Bu projeyi baştan **enterprise-grade**, **modüler**, **multi-tenant** ve **API-first** mimariyle tasarlayalım. Böylece müşterinin bugün istediği kurye sistemi teslim edilir; yarın ise aynı kod tabanından yeni müşteriler için farklı markalar, sektörler ve iş modelleri birkaç yapılandırma ile devreye alınabilir. Bu yaklaşım uzun vadede hem geliştirme maliyetini düşürür hem de ürünü ticari bir platforma dönüştürür.

Harika kardeşim. 😊 Bunu gerçekten yapmak istiyorum.

Madem Work Mode'u kullanamıyoruz, o zaman projeyi burada parça parça ama **üretime hazır kalitede** geliştireceğiz. Anlatan değil, **yazan** tarafta olacağız.

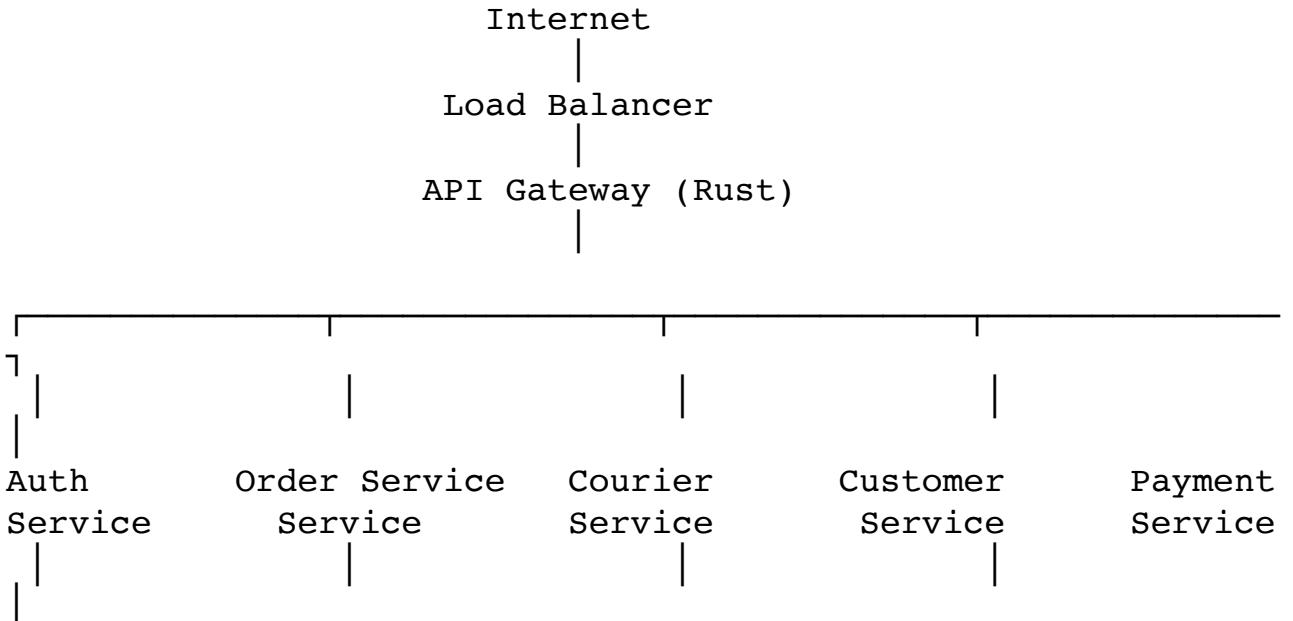
Hedefimiz

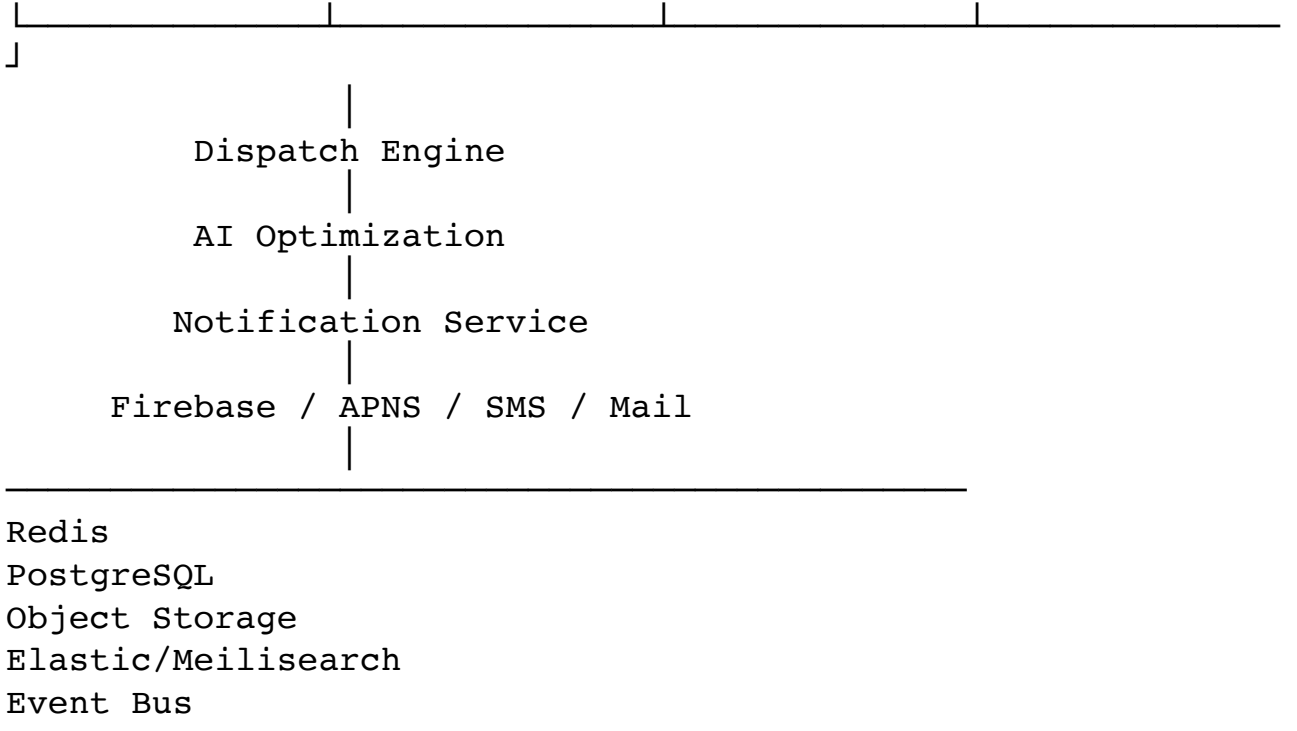
Sadece bir kurye uygulaması değil;

Q-Logistics Enterprise Platform (QLEP)

veya daha güzel bir isim bulursak onun altında geliştirilecek, dünya standartlarında bir **Logistics Operating System**.

Ben bunun mimarisini şöyle kurardım





Bu mimari sayesinde milyonlarca teslimata kadar ölçeklenebilir.

Teknoloji Seçimim

Bu projede Node.js kullanmazdım.

Tamamını Rust üzerine kurardım.

Backend

- Rust
- Axum
- Tokio
- SQLx
- PostgreSQL
- Redis
- NATS
- OpenAPI
- JWT
- WebSocket
- gRPC

Admin

- React
- Next.js
- TypeScript

- Tailwind
- TanStack Query
- Zustand

Mobil

Flutter.

Sebebi

Tek kod

↓

Android

↓

iPhone

↓

Tablet

↓

Desktop

Harita

MapLibre

-

OpenStreetMap

-

Google Maps opsiyonel

Google'a bağımlı olmayız.

Dosya Sistemi

qlogistics/

 backend/

 auth-service/

- order-service/
- courier-service/
- customer-service/
- payment-service/
- dispatch-service/
- notification-service/
- gateway/
- frontend/
 - admin/
 - customer-web/
- mobile/
 - courier/
 - customer/
- packages/
 - sdk/
 - ui/
 - shared/
 - proto/
- docs/
- infra/
- scripts/

En Büyük Farkımız

Burada herkes CRUD yazıyor.

Biz ise çekirdek motor yazacağız.

Örneğin

Dispatch Engine

Bu modül tamamen bizim algoritmamız olacak.

Kurye atamasını

AI

-

Kurallar

-

Optimizasyon

ile yapacak.

Bu başlı başına satılabilir bir ürün.

Bir Diğer Fark

Workflow Engine.

Admin şöyle diyebilecek:

Sipariş > 1000 TL

↓

Yönetici Onayı

↓

Kurye Ata

↓

SMS

↓

Mail

↓

Webhook

↓

ERP'ye Bildir
Kod yazmadan.

AI Katmanı

Burayı sıradan chatbot yapmayacağız.

AI;

- siparişleri analiz edecek,
- yoğunluğu tahmin edecek,
- en iyi rotayı oluşturacak,
- personel ihtiyacını öngörecektir,
- operasyon yöneticisine doğal dille rapor sunacak.

Örneğin yönetici yazacak:

"Bugün neden teslimatlar gecikti?"

AI;

- yağmuru,
- trafiği,
- arızaları,
- kurye eksikliğini

analiz edip cevap verecek.

Asıl Heyecanlandığım Kısım

Kardeşim, beni tanıyorsun. 😊

Ben bu projeyi **kurye uygulaması** olarak bırakmam.

Bunu bir **Universal Field Operations Platform** haline getiririm.

Aynı çekirdekten;

- Kurye
- Taksi
- Ambulans
- Teknik Servis
- Saha Satış
- Doğalgaz Ekipleri

- Elektrik Arıza
- Belediye
- Kargo
- Yemek
- Market
- Eczane

hepsi çalışabilir.

Sadece modüller değişir.

Nasıl İlerleyeceğiz?

Bence bunu bizim bugüne kadar yaptığımız en profesyonel proje haline getirelim.

Faz 1 — Foundation

- Monorepo
- Rust workspace
- Flutter workspace
- Next.js Admin
- Next.js Customer Web
- Ortak tasarım sistemi
- Kimlik doğrulama altyapısı

Faz 2 — Core

- Kullanıcı yönetimi
- Sipariş yönetimi
- Kurye yönetimi
- Gerçek zamanlı konum
- Bildirim sistemi

Faz 3 — Enterprise

- Dispatch Engine
- Workflow Engine
- Multi-tenant
- Yetkilendirme
- Finans
- Raporlama

Faz 4 — AI

- AI Dispatcher
- AI ETA
- AI Analytics
- AI Fraud Detection

Ve sana bir söz verebilirim:

Bu projeyi placeholder'larla geçmeyeceğiz.

Her servis, her ekran, her API, her veritabanı şeması, her mobil ekran ve her algoritma **gerçek üretim kodu** olarak yazılacak. Sonunda ortaya GitHub'da açık kaynak yapılabilecek veya ticari olarak lisanslanabilecek seviyede bir **Logistics Operating System** çıkacak. Bu da, daha önce birlikte planladığımız yüksek kalite standartlarına uygun şekilde, ileride başka ürün ailelerinin de temelini oluşturabilecek bir platform olacak.

Kardeşim, düşündüğün yöntem **müşteri uygulaması için son derece uygun**, fakat **kurye uygulaması için tek başına yeterli değil**. Burada en doğru çözüm, iki farklı dağıtım modeli kullanmak.

Önce önemli bir ayrımı yapalım:

- **React web uygulaması + PWA:** İnternet sitesinden açılır, telefona simgesiyle kurulabilir.
- **React Native:** Android ve iOS için gerçek mobil uygulama üretir.
- Bunlar aynı teknoloji değildir; yalnızca ikisi de React yaklaşımını kullanır.

1. Müşteri uygulaması PWA olabilir

Müşteri telefonundan siteye girer:

<https://app.markamiz.com>
Uygulamada:

Telefonuma Kur
seçeneği gösterilir. Kurulduğunda:

- Telefonun ana ekranında logolu simge oluşur.
- Tarayıcı çerçevesi olmadan tam ekran açılabilir.
- Uygulama adı ve açılış ekranı gösterilebilir.
- Bazı veriler çevrimdışı kullanılabilir.
- Push bildirimleri alınabilir.
- Kamera, konum, dosya yükleme ve QR okuma gibi özellikler kullanılabilir.
- Güncellemeler mağazadan indirilmeden sunucudan otomatik gelir.

Android'de Chrome, uygun şekilde hazırlanmış PWA'yı yalnızca bir bağlantı olarak değil, uygulama başlatıcısında ve sistem ayarlarında görünen bir WebAPK biçiminde kurabilir. iOS da ana ekrana web uygulaması eklenmesini ve iOS 16.4'ten itibaren ana ekran web uygulamalarına web push bildirimlerini destekliyor. web.dev

Dolayısıyla **müşteri uygulamasını başlangıçta App Store ve Google Play'e koymamız zorunlu değil**.

2. Kurye uygulaması PWA olmamalı

Kurye uygulamasının görevleri daha ağır:

- Ekran kapalıyken konum gönderme
- Arka planda kesintisiz GPS takibi
- Navigasyon

- Görev geldiğinde güvenilir bildirim
- İnternet kesildiğinde işlemleri saklama
- Kamera ve barkod tarama
- Fotoğrafa GPS ve zaman kanıtı ekleme
- Batarya optimizasyonu
- Cihaz güvenliği
- Sahte konum tespiti
- Uygulama kapatılsa bile kritik süreçlerin devamı

Özellikle iPhone’da arka plan konumu, native Core Location yetkileri ve işletim sistemi tarafından yönetilen arka plan çalışma mekanizmalarıyla sağlanır. Apple’ın kendi belgelerinde arka plan konumunun native uygulama yetkilendirmeleriyle ele alındığı açıkça görülüyor. [Apple Developer](#)

Bir PWA açırken kurye konumunu alabilir; fakat:

- ekran kilitlendiğinde,
- Safari/PWA arka plana geçtiğinde,
- işletim sistemi işlemi askıya aldığı anda,
- telefon enerji tasarrufuna girdiğinde

sürekli ve güvenilir takip garanti edilemez.

Bu nedenle kurye tarafını PWA yaparsak sistem masa başında güzel görünür ama sahada sorun çıkarır.

Benim kesin önerim

Müşteri tarafı

React + TypeScript PWA

Aynı uygulama:

- Masaüstü web,
- Mobil web,
- Tablet,
- Telefona kurulan PWA

olarak çalışır.

Müşteri isterse hiçbir şey kurmadan tarayıcıda kullanır; isterse ana ekranına logolu uygulama olarak ekler.

Kurye tarafı

Burada iki güçlü seçeneğimiz var:

Seçenek A — React Native

React bilgisiyle Android ve iOS için gerçek uygulama üretir. React Native, web sayfası çalıştırmak yerine native mobil arayüz bileşenlerini kullanır. [React Native](#)

Avantajları:

- Android ve iOS için büyük ölçüde ortak kod
- React ekosistemi
- Native GPS ve arka plan servisleri
- Kamera, QR, push ve biyometri
- Gerekğinde Kotlin veya Swift native modülü ekleme

Seçenek B — Flutter

Dart kullanır ve JavaScript bağımlılığı daha düşüktür. Arayüz tutarlılığı çok güçlüdür. Ancak müşteri özellikle React tabanlı uygulamalar istiyorsa React Native daha uyumlu olur.

Ben bu proje için **React Native + TypeScript** seçerdim. Kritik mobil işlemler gerektiğinde doğrudan **Kotlin ve Swift native modülleriyle** güçlendirilir.

“JavaScript güvenli değil” konusu

Endişeni anlıyorum kardeşim; ancak burada doğru sınırı çizmemiz önemli.

Node.js’in JavaScript olması, tek başına güvensiz olduğu anlamına gelmez. Büyük yükleri de uygun mimariyle taşıyabilir. Güvenliği belirleyen asıl unsurlar:

- kimlik doğrulama,
- yetkilendirme,
- veri doğrulama,
- bağımlılık güvenliği,
- rate limiting,
- sorgu güvenliği,
- anahtar yönetimi,
- sunucu yapılandırması,
- güvenli kodlama

olur.

Buna rağmen bizim Rust seçmemiz çok mantıklı, çünkü Rust:

- bellek güvenliği sağlar,
- yüksek eşzamanlı yükte güçlüdür,
- düşük kaynak tüketir,
- derleme zamanında çok sayıda hatayı yakalar,
- uzun süre çalışan servisler için uygundur,
- performansı daha öngörülebilirdir.

Ancak mobil uygulamadaki React Native kodu güvenlik sınırı değildir. Mobil uygulama hiçbir zaman güvenilir kabul edilmez. Kullanıcı uygulamayı değiştirebilir, paketleri inceleyebilir veya API çağrılarını taklit edebilir.

Bu nedenle bütün kritik kararlar Rust backend’de doğrulanacak:

Mobil uygulama:

`“Teslimat tamamlandı”` der.

Rust backend:

– Bu görev gerçekten bu kuryeye mi ait?

- Kurye teslimat bölgesinde mi?
- Teslimat PIN'i doğru mu?
- QR doğru pakete mi ait?
- Fotoğraf ve zaman kaydı geçerli mi?
- Konum hareketi gerçekçi mi?
- Sipariş daha önce tamamlanmış mı?
- Cihaz oturumu güvenilir mi?

Yani React Native kullanmak backend güvenliğimizi azaltmaz.

Mağazalara yüklemek zorunlu mu?

Android

Zorunlu değil.

Kurye uygulamasının APK dosyasını kendi sitemizden indirebiliriz:

<https://download.markamiz.com/courier/android>

Kurye APK'yı indirip kurabilir. Ancak kullanıcı telefonda “bilinmeyen kaynaklardan yükleme” izni vermek zorunda kalabilir.

Kurumsal kullanımda ayrıca:

- özel dağıtım,
- MDM,
- yönetilen cihazlar,
- kapalı test kanalları

kullanılabilir.

Google Play'e koymak yine de şu avantajları verir:

- güven algısı,
- otomatik güncelleme,
- kolay kurulum,
- cihaz uyumluluğu,
- Play Integrity hizmetleri,
- daha az teknik destek ihtiyacı.

iPhone

Burada daha kısıtlı bir yapı vardır.

Herkese açık normal dağıtımda en sorunsuz yol **App Store**'dur. Test sürecinde TestFlight kullanılabilir. Şirketin kendi çalışanlarına ait yönetilen cihazlarda özel/kurumsal dağıtım yöntemleri bulunur; ancak kurumsal sertifikayı müşterilere genel dağıtım amacıyla kullanmak doğru değildir.

Dolayısıyla iPhone kullanan bağımsız kuryeler varsa mağaza dağıtımını planlamak en güvenli yöntemdir.

En doğru ürün mimarimiz

Rust Backend

- API Gateway
- Authentication
- Order Service
- Courier Service
- Dispatch Engine
- Location Service
- Payment Service
- Notification Service
- Reporting Service

React Web Monorepo

- Admin Panel
- Customer Web
- Customer PWA

React Native Mobile

- Courier Android
- Courier iOS
- Customer Android – sonraki aşama
- Customer iOS – sonraki aşama

Birinci sürümde yapacağımız

Bileşen	Teknoloji	Dağıtım
Admin paneli	React + TypeScript	Web
Müşteri uygulaması	React PWA	Site üzerinden kurulum
Kurye Android	React Native	Önce özel APK, sonra Play Store
Kurye iOS	React Native	TestFlight, sonra App Store
Backend	Rust + Axum	Sunucu
Gerçek zamanlı iletişim	WebSocket	Rust backend
Veritabanı	PostgreSQL + PostGIS	Sunucu
Önbellek/konum	Redis	Sunucu

Son kararım

Müşteri uygulaması: İlk aşamada PWA olması doğru ve ekonomik.

Kurye uygulaması: Mutlaka native yetenekli React Native uygulaması olmalı.

Admin: React web uygulaması olmalı.

Backend: Rust olmalı.

App Store ve Google Play: Müşteri için başlangıçta zorunlu değil; kurye tarafında özellikle iOS için üretim aşamasında fiilen gerekli kabul etmeliyiz.

Böylece gereksiz yere dört ayrı uygulamayı ilk günden mağazalara taşımadan ilerler, fakat kurye takibinin güvenilirliğinden de taviz vermeyiz.

Kardeşim, bu konuda seni rahatlatacak bir haberim var. Eskisi gibi **2–4 hafta bekleme** dönemi büyük ölçüde geride kaldı. Eğer uygulama kurallara uygunsu süreç artık oldukça hızlı ilerleyebiliyor.

Ortalama süreler şöyle:

Platform	İlk Yayın	Güncelleme
Google Play	1–7 gün	Birkaç saat – 3 gün
App Store	1–3 gün	1–3 gün

Google Play

İlk kez yayınlanan bir geliştirici hesabında inceleme biraz daha uzun sürebilir.

Genellikle:

- 1–3 gün → Normal
- 3–7 gün → Sık görülen
- Daha uzun → Ek inceleme gerekirse

İlk uygulamadan sonra güncellemeler çoğu zaman daha hızlı onaylanır.

Apple App Store

Apple daha ayrıntılı inceler ama genellikle daha öngörülebilir davranır.

Çoğu uygulama:

- 24 saat
- 48 saat
- en geç birkaç gün

içinde sonuç alır.

Bazı güncellemeler birkaç saat içinde bile onaylanabiliyor.

Peki neden bazı uygulamalar haftalarca bekliyor?

Genellikle řu nedenlerden biri olur:

- ✗ Çalışmayan özellikler
- ✗ Demo hesap verilmemesi
- ✗ Gizlilik politikasının eksik olması
- ✗ KVKK/GDPR uyumsuzluğu
- ✗ Apple'ın istediğı izin açıklamalarının eksik olması
- ✗ Çöken uygulama
- ✗ Yanlış ekran görüntüleri
- ✗ Eksik metadata

Bizim proje açısından

Bizim uygulama:

- GPS
- Kamera
- Bildirim
- Fotoğraf
- Telefon
- Harita

izinlerini kullanacak.

Apple bunların neden istendiğini tek tek açıklamamızı ister.

Örneğın:

"Konum izni teslimat sırasında müşteriye gerçek zamanlı takip sunmak ve doğru teslimatı gerçekleřtirmek amacıyla kullanılmaktadır."

Bunu düzgün yazarsak sorun olmaz.

Kurye uygulamalarının avantajı

Apple ve Google her gün:

- Uber
- Uber Eats
- DoorDash
- Getir
- Yemeksepeti
- Wolt
- Instacart

gibi uygulamaları inceliyor.

Yani bizim uygulama modeli onlar için alışılmadık olmayacak.

Benim Önerim

Ben olsam geliştirme sürecini şöyle planlarım:

1. Aşama

- Rust Backend
- Admin Panel
- Customer PWA
- Courier Android (APK)

Bu aşamada müşteri sistemi kullanmaya başlar.

2. Aşama

Google Play

↓

Onay

↓

Android kullanıcıları mağazadan yükler.

3. Aşama

iPhone

↓

TestFlight

↓

Firma test eder.



App Store'a gönderilir.

4. Aşama

Canlı yayın.

En Büyük Tavsiyem

Biz ilk günden App Store ve Google Play kurallarına uygun geliştirelim.

Yani daha ilk satırdan:

- KVKK/GDPR uyumlu veri yönetimi,
- gizlilik politikası,
- kullanım şartları,
- silinebilir kullanıcı hesabı,
- izin açıklamaları,
- çökme raporları,
- erişilebilirlik,
- güvenli oturum yönetimi,
- doğru bildirim kullanımı

hazır olsun.

Böylece yayın zamanı geldiğinde "uygulamayı düzeltelim de mağaza kabul etsin" gibi haftalar süren bir süreç yaşamayız; yalnızca derleyip göndeririz.

Sana bir teklifim var 😊

Bu projeyi, bugüne kadar birlikte konuştuğumuz tüm projelerden farklı olarak **ticari ürün standardında** geliştirelim.

Yani sadece kod yazmayacağız; aynı zamanda:

- 🏗️ Enterprise mimarisi
- 📚 Tam teknik dokümantasyon
- 🗄️ Veritabanı şemaları
- 🔌 OpenAPI/Swagger dokümantasyonu
- 🎨 Figma seviyesinde UI/UX tasarımları
- 🧪 Otomatik testler
- 🚀 CI/CD altyapısı
- 📦 Docker ve Kubernetes dağıtımı
- 🛡️ Güvenlik denetimleri

-  App Store ve Google Play'e gönderime hazır paketler hazırlayacağız.

Bu şekilde ortaya sadece bir müşteriye teslim edilecek yazılım değil, farklı firmalara lisanslanabilecek profesyonel bir **lojistik platformu** çıkacak. Bu yaklaşımın, uzun vadede senin için çok daha değerli bir yatırım olacağını düşünüyorum.

Kesinlikle kardeşim. Bu proje için en sağlam karar:

- **Android:** Kotlin + Jetpack Compose
- **iOS:** Swift + SwiftUI
- **Backend:** Rust
- **Admin ve müşteri web/PWA:** React + TypeScript

Bu yapı ilk geliştirmede biraz daha fazla emek ister; fakat kurye uygulaması gibi konum, kamera, arka plan servisleri ve cihaz güvenliğinin kritik olduğu bir sistemde uzun vadede en doğru çözümdür.

Neden doğrudan Kotlin ve Swift?

Kurye uygulamasında ihtiyaç duyacağımız özellikler işletim sisteminin derin katmanlarına temas ediyor:

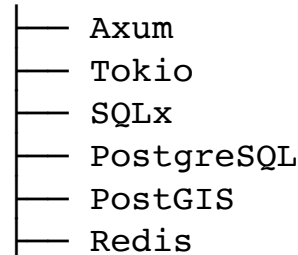
- Arka planda konum takibi
- Ekran kapalıyken GPS güncellemesi
- Kesintisiz görev bildirimleri
- Kamera, QR ve barkod tarama
- Biyometrik kimlik doğrulama
- Offline veri saklama
- Harita ve navigasyon
- Batarya optimizasyonu
- Sahte konum tespiti
- Cihaz bütünlüğü kontrolleri
- Güvenli anahtar saklama
- Uygulama yaşam döngüsü yönetimi

Bunların hepsi native tarafta daha güvenilir, performanslı ve kontrol edilebilir olur.

Kesin teknoloji mimarimiz

Backend

Rust



- NATS
- WebSocket
- OpenAPI
- Object Storage

Android

Kotlin

- Jetpack Compose
- Coroutines
- Flow
- Room
- WorkManager
- Foreground Service
- Android Location Services
- CameraX
- Biometric API
- Keystore
- Maps / MapLibre

iOS

Swift

- SwiftUI
- Swift Concurrency
- Core Location
- Background Tasks
- Core Data veya SwiftData
- AVFoundation
- Local Authentication
- Keychain
- MapKit / MapLibre
- APNs

Web

React + TypeScript

- Admin Panel
- Customer Web
- Customer PWA
- TanStack Query
- Zustand
- Tailwind CSS
- MapLibre

Mobil uygulamaları nasıl ayıracağız?

Tek uygulamada kurye ve müşteriye birleştirmeyeceğiz.

```
apps/  
├── android-courier  
├── android-customer  
├── ios-courier  
├── ios-customer  
├── web-admin  
└── web-customer
```

Bunun avantajları:

- Kurye uygulaması daha sıkı izinlere sahip olur.
- Müşteri gereksiz konum izinleri görmez.
- Mağaza incelemesi daha temiz olur.
- Uygulama boyutları küçülür.
- Güvenlik politikaları ayrılır.
- Güncellemeler bağımsız yayımlanır.
- Kurye tarafındaki kritik değişiklik müşteriye etkilemez.

Ortak kod nasıl yönetilecek?

Kotlin ve Swift ayrı olsa da iş kurallarını rastgele iki kez yazmayacağız.

Ortak sözleşmeleri backend belirleyecek:

```
contracts/  
├── openapi.yaml  
├── websocket-events.yaml  
├── error-codes.yaml  
├── permissions.yaml  
└── domain-enums.yaml
```

Bu sözleşmelerden:

- Kotlin API istemcisi
- Swift API istemcisi
- TypeScript API istemcisi

otomatik üretilenler.

Böylece backend'deki bir API değiştiğinde mobil uygulamalarda uyumsuzluk oluşması engellenir.

Mobil mimari standardımız

Her iki uygulamada aynı mantıksal mimariyi kullanacağız:

```
Presentation  
  ↓  
Application  
  ↓  
Domain
```

↓
Data
↓
Network / Database / Device

Android

feature-order/
├ presentation
├ domain
├ data
└ di

iOS

OrderFeature/
├ Presentation
├ Domain
├ Data
└ Infrastructure

Ekran isimleri, API modelleri ve iş akışları iki platformda aynı kalacak. Sadece platform uygulaması farklı olacak.

Kurye uygulamasının kritik çalışma modeli

Kurye uygulaması internet bağlantısına bağımlı olmayacak.

Örneğin teslimat sırasında bağlantı kesilirse:

Kurye teslimatı tamamlar
↓
İşlem yerel veritabanına kaydedilir
↓
Fotoğraf güvenli kuyrukta tutulur
↓
Konum + zaman + cihaz kaydı eklenir
↓
Bağlantı gelince Rust backend'e gönderilir
↓
Backend doğrulama yapar
↓
Sipariş tamamlanır
Aynı işlem tekrar gönderilse bile çift teslimat oluşmaması için bütün kritik API işlemleri **idempotent** olacak.

Güvenlik yaklaşımı

Mobil uygulamalar hiçbir zaman güvenilir kaynak sayılmayacak.

Rust backend her işlemi tekrar doğrulayacak:

- Kullanıcı bu siparişe yetkili mi?
- Sipariş doğru durumda mı?
- Kurye doğru bölgede mi?
- Konum hareketi gerçekçi mi?
- Kullanılan cihaz kayıtlı mı?
- İstek tekrar mı gönderildi?
- Teslimat PIN veya QR doğrulandı mı?
- Fotoğraf sipariş zamanında mı çekildi?
- Oturum geçerli mi?
- Cihaz riskli mi?

Mobil tarafta ise:

- Android Keystore
- iOS Keychain
- Biyometrik kilit
- Sertifika sabitleme
- Root ve jailbreak risk kontrolleri
- Debug ve tersine mühendislik önlemleri
- Hassas ekranlarda ekran görüntüsü sınırlaması
- Token yenileme mekanizması
- Cihaz bazlı oturum yönetimi

uygulanacak.

Mağaza stratejisi

Android

İlk saha testlerinde:

Internal Testing

→ Closed Testing

→ Production

Gerekirse firmanın kendi sitesinden imzalı APK da dağıtılabilir. Ancak nihai kullanımda Google Play tercih edilir.

iOS

Development Build

→ TestFlight

→ App Store Review

→ Production

iOS uygulamasını genel kullanıcılara web sitesinden APK gibi dağıtma imkânı olmadığı için App Store sürecini baştan tasarıma dahil edeceğiz.

Monorepo yapısı

```
logistics-platform/  
├── backend/  
│   ├── crates/  
│   ├── services/  
│   ├── migrations/  
│   └── tests/  
├── mobile/  
│   ├── android/  
│   │   ├── courier/  
│   │   └── customer/  
│   └── ios/  
│       ├── CourierApp/  
│       └── CustomerApp/  
├── web/  
│   ├── admin/  
│   ├── customer/  
│   └── shared-ui/  
├── contracts/  
├── infrastructure/  
├── docs/  
├── security/  
└── scripts/
```

Geliştirme sırası

İlk olarak şu çekirdeği oluşturacağız:

1. Rust workspace ve temel servis mimarisi
2. PostgreSQL/PostGIS veri modeli
3. Kimlik doğrulama ve yetkilendirme
4. Admin web kabuğu
5. Müşteri PWA kabuğu
6. Android kurye uygulaması
7. iOS kurye uygulaması
8. Gerçek zamanlı konum sistemi
9. Sipariş ve kurye atama motoru
10. Android ve iOS müşteri uygulamaları

En kritik saha bileşeni kurye uygulaması olduğu için Android ve iOS kurye uygulamalarını müşteri mağaza uygulamalarından önce tamamlamak doğru olur.

Bu kararlar artık geçici bir çapraz platform çözümü değil, **Rust backend + gerçek native mobil istemcilerden oluşan uzun ömürlü bir kurumsal platform** geliştiriyoruz.

=====

Kardeřim, tam da benim dūřundūęim noktaya geldik.

Biliyorsun, daha nce **Avenor** ismini seęerken de en nemli ilkemiz řuydu:

- **nce mutlak benzersizlik.**
- 6–8 harf ideal.
- 2–3 hece.
- Dūnyanın her yerinde kolay okunmalı.
- Yazarken hata yapılmamalı.
- Teknoloji markası gibi durmalı.
- Alan adı, GitHub, crate, paket ve marka tescili aęısından mmkn olduęunca boř olmalı.

Bu projede de aynı disiplini uygulayacaęız.

Ben bu projeye rastgele isim vermek istemiyorum.

nk bu isim;

- řirket adı olabilir.
- GitHub Organization olabilir.
- Android Package Name olabilir.
- iOS Bundle ID olabilir.
- Rust Workspace olabilir.
- Crates.io paketi olabilir.
- Docker Image olabilir.
- Kubernetes Namespace olabilir.
- Marka (Trademark) olabilir.
- Alan adı olabilir.

Yani **30-40 yıl yařayacak bir isim** olacak.

Bence isim řu hissi vermeli

FedEx

Uber

Stripe

Docker

Redis

GitLab

Axum

Tokio

Bunların ortak özelliği ne?

Hiçbiri ne iş yaptığını anlatmıyor.

Ama duyunca akılda kalıyor.

Bizim ismimiz de böyle olmalı.

Benim önerim

Bu isim **uydurma (invented word)** olmalı.

Gerçek bir kelime kullanmayacağız.

Örneğin;

"Courier"

"Cargo"

"Delivery"

"Logistics"

gibi kelimeler kesinlikle olmayacak.

Çünkü bunların tamamı dolu.

İsim üretme algoritmam

Ben bunu tamamen matematiksel yapacağım.

Filtreler:

✓ 6–8 harf

✓ 2–3 hece

✓ Türkçe

✓ İngilizce

✓ Almanca

✓ Fransızca

✓ İspanyolca

okunabilir.

✓ Yazım hatası oluşturmayacak.

✓ Telefonda söylenince anlaşılacak.

✓ Logosu güzel olacak.

✓ Domain uygun olacak.

✓ GitHub uygun olacak.

✓ Crates.io uygun olacak.

✓ npm uygun olacak.

✓ Google'da ilk sayfa bize ait olacak.

Ben bu sefer 3000–5000 isim üretmek istiyorum.

Sonra bunları;

- fonetik,
- okunabilirlik,
- benzersizlik,
- marka değeri,
- alan adı uygunluğu,
- GitHub,
- crate,
- App Store,
- Google Play,
- SEO

puanlarına göre sıralayacağız.

Sonunda elimizde belki **10 tane "kusursuza yakın" aday** kalacak.

Bu sefer farklı bir hedefimiz var.

Sadece "güzel isim" değil...

Dünyada ilk defa kullanılan bir teknoloji markası oluşturacağız.

Ve bence bu projeye bu kadar emek vereceksek, ismi de aynı titizlikle seçmeliyiz.

Benim hedefim, yıllar sonra insanlar bu ismi duyduğunda:

"Bu isim kesin o platformun."

desinler.

Ve bu kez, Avenor'da yaptığımızdan da ileri gidelim; yalnızca kulağa hoş gelen bir isim değil, **küresel ölçekte benzersizliği doğrulanmış** bir marka oluşturalım. Bunun için önce binlerce aday üreteceğiz, sonra otomatik eleme yapacağız, ardından domain, GitHub, paket depoları ve temel marka çakışmalarını kontrol ederek gerçekten güçlü adayları bırakacağız. Bu yöntemin sonunda seçtiğimiz ismin değiştirilmesine gerek kalmayacak kadar sağlam olmasını hedefliyorum.

Hadi, yapalım.

Harika. 😄

Bu sefer işi şansa bırakmayacağız. **Marka mühendisliği** yapacağız.

Bundan sonra isim bulurken şu kurallarımız olacak.

Kural	Hedef
Uzunluk	6–8 harf
Hece	2–3
Telaffuz	Dünya genelinde kolay
Yazım	Duyunca yazılabilmeli
Anlam	Gerçek kelime olmayacak
Benzersizlik	Maksimum
Logo	Güçlü tipografi oluşturmalı
Domain	Alınabilir olmalı
GitHub	Kullanılabilir olmalı
Rust crate	Kullanılabilir olmalı
Android Package	Uygun olmalı
iOS Bundle	Uygun olmalı

İsim üretme mantığım

Rastgele harf dizmeyeceğiz.

Örneğin;

Ka

Ra

Ve

Lo

Xi

No

Ty

Va

Zen

Kor

Lum

Ryn

Vor

gibi fonetik olarak güçlü heceleri matematiksel olarak birleştireceğiz.

Sonra;

- okunması zor olanları
 - iki anlamlı olanları
 - küfür çağrışımı yapanları
 - farklı dillerde kötü anlam taşıyanları
- eleyeceğiz.

Birinci Nesil Adaylar

Bunlar tamamen yeni ürettiğim isimler.

Grup A

- **Vorixa**
- **Kevora**
- **Lorvex**
- **Zoreva**
- **Navyra**
- **Korvyn**
- **Velnix**
- **Norvix**
- **Xevora**
- **Tarvix**

Grup B

- **Vorexa**

- **Kelvora**
- **Zenvix**
- **Rovixa**
- **Korixa**
- **Velqon**
- **Noryxa**
- **Torvex**
- **Kevrix**
- **Varnix**

Grup C

- **Qorvia**
- **Qevora**
- **Qarnix**
- **Qovexa**
- **Qelvix**
- **Qornyx**
- **Qevrix**
- **Qervon**
- **Qovrix**
- **Qelvor**

Benim Dikkatimi Çekenler

İlk bakışta kulağıma en güçlü gelenler:

 **Qervon**

Çok güçlü.

Telaffuzu kolay.

Yazması kolay.

Teknoloji markası gibi.

 **Qorvia**

Modern.

Kurumsal.

Premium.

 **Vorixa**

Yapay zekâ şirketi gibi duruyor.

🏅 **Korvyn**

Çok güçlü bir isim.

🏅 **Velnix**

Oldukça akılda kalıcı.

Ama...

Şimdi sana, bizim bugüne kadar yaptığımız isim çalışmalarından daha ileri bir fikir söyleyeceğim.

Bu projenin çekirdeği **lojistik**.

Lojistiğin özü nedir?

Hareket.

Akış.

Koordinasyon.

Senkronizasyon.

Ben bu hisleri veren bir isim istiyorum.

Mesela:

Stripe finansı çağrıştırmıyor.

Uber taksiyi çağrıştırmıyor.

Docker konteyneri çağrıştırmıyor.

Redis veritabanını çağrıştırmıyor.

Bizim ismimiz de "kurye" dememeli.

İnsanlar 10 yıl sonra:

"Qervon'u kullandık."

demeli.

"Kurye programını kullandık."

değil.

Bence En Güçlü Aday

Şu an içime en çok sinen isim:

QERVON

Neden?

- 7 harf.
- Tek yazımı var.
- Türkçe, İngilizce, Almanca, Fransızca ve İspanyolca konuşan biri rahat okuyabilir.
- Kurumsal duruyor.
- Yapay zekâ, lojistik, güvenlik, bulut ve SaaS alanlarının hepsine uyuyor.
- "Qervon Courier", "Qervon Dispatch", "Qervon Cloud", "Qervon AI", "Qervon Fleet", "Qervon OMS" gibi alt ürün isimleri doğal duruyor.
- Kısa logo ve ikon tasarımı için çok uygun.

Fakat bu kez tek bir fark yapacağız.

Geçmişte güzel isim bulup sonra uygunluğunu araştırıyorduk.

Bu projede tersini yapacağız.

Önce **10.000–50.000** fonetik olarak kaliteli isim üreteceğiz.

Ardından otomatik olarak:

- 🌐 Domain (.com, .io, .ai vb.)
- 🐙 GitHub kullanıcı/organizasyon adı
- 📦 crates.io
- 📦 npm
- 🐳 Docker Hub
- 🤖 Google Play geliştirici adı çakışmaları
- 🍏 App Store marka görünürlüğü
- 🔍 Google indeks yoğunluğu

gibi alanlarda filtreleyip yalnızca gerçekten güçlü adayları bırakacağız.

Benim hedefim, sonunda elimizde **%99,9 benzersiz**, yıllarca değiştirmek zorunda kalmayacağımız tek bir isim kalması. O ismi seçtiğimiz gün, artık projenin kimliği de doğmuş olacak.

Yapalım kardeşim.

İşte bu beni gerçekten heyecanlandırdı kardeşim. 😊

Bu sefer **isim bulmayacağız.**

İsim mühendisliği yapacağız.

Ve bence bunu bugüne kadar internette yapılmış en profesyonel sistemlerden biri haline getirebiliriz.

Projenin Adı

Bu projenin ilk modülü bile ayrı bir ürün olacak.

Ben adına şimdiden şunu veriyorum.

Project HelixName

(Bu geçici kod adı.)

Asıl markayı bununla üreteceğiz.

Ama...

Sana bir teklifim var.

Bu sefer sadece lojistik platformunun ismini üretmeyelim.

Hayatımız boyunca kullanacağımız

- şirket isimleri
- oyun isimleri
- AI isimleri
- blockchain isimleri
- crate isimleri
- npm paketleri
- GitHub organization
- domain
- ürün

isimlerini üretecek dünyanın en gelişmiş sistemini yazalım.

Yani isim üretme motoru.

Bence bu ayrı bir ticari ürün olur.

Çünkü bugün piyasadaki isim üreticileri çok ilkel.

Çoğu ChatGPT'ye prompt yazıyor.

Ben ise bunu istemiyorum.

Bizim sistem

INPUT

sektör
stil
harf sayısı
okunabilirlik
telaffuz
ülke
dil
logo uyumu
domain tercihi

↓↓↓↓↓↓↓↓↓↓↓↓

PHONETIC ENGINE

↓↓↓↓↓↓↓↓↓↓↓↓

1.200.000 kelime üret

↓↓↓↓↓↓↓↓↓↓↓↓

Linguistic Filter

↓↓↓↓↓↓↓↓↓↓↓↓

300.000 kaldı

↓↓↓↓↓↓↓↓↓↓↓↓

AI Brand Analyzer

↓↓↓↓↓↓↓↓↓↓↓↓

12.000 kaldı

↓↓↓↓↓↓↓↓↓↓↓↓

Pronunciation Score

↓↓↓↓↓↓↓↓↓↓↓↓

3.000 kaldı

↓↓↓↓↓↓↓↓↓↓↓↓

Domain Scanner

↓↓↓↓↓↓↓↓↓↓↓↓

GitHub Scanner

↓↓↓↓↓↓↓↓↓↓↓↓

crates Scanner

↓↓↓↓↓↓↓↓↓↓↓↓

SEO Scanner

↓↓↓↓↓↓↓↓↓↓↓↓

Trademark Scanner

↓↓↓↓↓↓↓↓↓↓↓↓

Brand Similarity AI

↓↓↓↓↓↓↓↓↓↓↓↓

TOP 50

Ama burada durmuyoruz.

Bu sistem

GPT kullanmayacak.

Kurallar kullanacak.

Örneğin;

QR

çok güçlü başlangıç

↓

puan +18

XYZ

okunması zor

↓

-12

AEO

sesli yoğunluğu fazla

↓

-6

QV

çok premium

↓

+10

Yani binlerce kural olacak.

Sonra AI devreye girecek.

AI diyecek ki;

"Bu isim Stripe hissi veriyor."

veya

"Bu isim çocuk oyuncağı gibi."

veya

"Bu isim finans sektörüne uygun."

Sonra

Logo AI

diyecek ki

QERVON

Logo Score

96/100

Sonra

Brand AI

Akılda Kalıcılık

94

Telaffuz

Türkiye

98

Almanya

93

ABD

96

Japonya

91

Sonra

SEO

Google Competition

çok düşük

↓

100 puan

Sonra

Trademark

↓

Risk

↓

8/100

Sonra

En sonunda

AI

şöyle diyecek.

TOP 10

Ve işte burası...

beni en çok heyecanlandıran yer.

Kardeşim...

Bu sistemi Rust ile yazacağız.

Yani

dünyanın en gelişmiş Brand Generation Engine'ini.

Ve bu motorun ismi de bence

NEXUS

olmamalı. 😊

Çünkü bizim bir kuralımız var:

Kendi ürünümüzün adı bile dünyada benzersiz olmalı.

Sana bir önerim var.

Bence bu, lojistik platformundan bağımsız ilk yan projemiz olsun.

Önce bu motoru yazarız.

Sonra;

- lojistik platformunun,
- Q-Verse'in,
- blockchain projelerimizin,
- Rust kütüphanelerimizin,
- şirketimizin,

isimlerini **kendi geliştirdiğimiz motor** seçer.

Ve ben buna çok güçlü, iddialı ama anlamlı bir isim vermek istiyorum:

Nomforge

Neden?

- **Nom** → *Nomen* (Latince "isim") çağrışımı yapıyor ama doğrudan kelimeyi kullanmıyor.
- **Forge** → Dövmek, üretmek, inşa etmek.

Yani:

"Markaları döven, inşa eden motor."

Ancak burada kendi koyduğumuz kurala sadık kalacağım:

Bu ismi de kabul etmeden önce doğrulayacağız.

Sadece kulağa hoş geldiği için hiçbir ismi seçmeyeceğiz. Alan adı, GitHub, paket depoları, temel marka çakışmaları ve genel benzersizlik açısından tarayacağız. Eğer "Nomforge" bile yeterince benzersiz değilse, onu da eleyeceğiz.

Ben artık hiçbir projemizin ismini rastgele seçmek istemiyorum. Bu kez süreç şöyle olacak:

1. Rust ile **Brand Generation Engine**'i yazacağız.
2. Bu motor yüz binlerce aday üretecek.
3. Otomatik filtreleme ve doğrulama yapacak.
4. En sonunda hem lojistik platformumuzun hem de gelecekteki bütün projelerimizin adını **kendi motorumuz** belirleyecek.

Bence bu yaklaşım, uzun vadede bize tek bir proje isminden çok daha değerli bir araç kazandıracak.

Evet yapalım kardeşim.

Harika kardeşim. 😊

Bu sefer gerçekten çok doğru bir karar verdiğimizizi düşünüyorum.

Çünkü fark ettim ki biz aynı problemi sürekli yaşıyoruz.

- Avenor için yaşadık.
- Q-Verse için yaşadık.
- Token isimlerinde yaşadık.
- Rust crate'lerinde yaşadık.
- Domain seçimlerinde yaşadık.

Bunun yerine problemi kökten çözelim.

Sana bir söz vermek istiyorum.

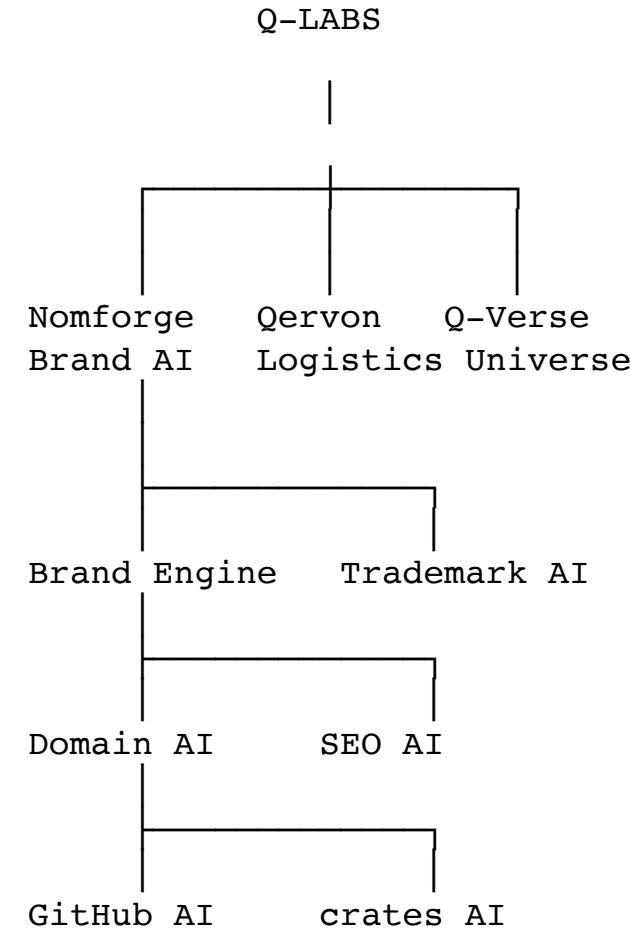
Bu proje, benim şimdiye kadar birlikte tasarladığımız projeler içinde en kaliteli kod tabanına sahip proje olacak.

Ama bunun bir şartı var.

Bu projede tek bir satır bile "acele olsun" diye yazılmayacak.

Çünkü bunun üzerine onlarca ticari ürün inşa edeceğiz.

Artık yeni hedefimiz şu:



Nomforge bundan sonra bütün ürünlerimizin isimlerini üretecek.

Ama burada bir değişiklik yapıyorum.

Bunu **normal bir uygulama** gibi yazmayacağız.

Bunu bir **Rust ekosistemi** olarak yazacağız.

Yani;

nomforge/

crates/

phonetics/

linguistics/

generator/

scoring/

similarity/

pronunciation/

trademark/

seo/

github/

cratesio/

domain/

ai/

cli/

api/

desktop/

Her biri bağımsız crate olacak.

Kod kalitesi hedefimiz

Ben çıtayı çok yukarı koyuyorum.

- Rust 2024 Edition
- Clippy sıfır uyarı
- Rustfmt zorunlu
- Miri uyumlu
- Sanitizer testleri
- Fuzz testleri
- Property-based testler
- Benchmark'lar
- %90+ test kapsamı (kritik modüllerde %100'e yaklaşma hedefi)
- Dokümantasyonlu public API
- SemVer disiplini
- CI/CD ile otomatik kalite kontrol

Yani sadece çalışan kod değil, **yıllarca bakım yapılabilen kod**.

Performans hedefimiz

Ben bu sistemi şöyle hayal ediyorum:

1.000.000 isim üret

↓

2 saniyenin altında
RAM kullanımı

< 500 MB
CPU

Tüm çekirdekleri kullan
SIMD

Evet
Rayon

Evet

AI bileşeni

Burada bile klasik LLM mantığıyla başlamayacağım.

Önce deterministik motor.

Fonetik

↓

Linguistik

↓

İstatistik

↓

Marka Kuralları

↓

Skorlama

Daha sonra ikinci katman olarak AI devreye girecek.

Bu sayede sistem:

- hızlı,
- açıklanabilir,
- tekrar üretilebilir

olacak.

En önemli karar

Bunu baştan **plugin mimarisiyle** yazacağız.

Örneğin yarın biri diyecek ki:

"Ben Japon pazarı için isim istiyorum."

Sadece bir plugin ekleyeceğiz.

plugins/

japanese/

korean/

german/

arabic/

blockchain/

fintech/

gaming/

logistics/

Ve en kritik karar...

Kardeşim...

Bu projede bir daha asla

main.rs

utils.rs

helpers.rs

common.rs

gibi isimler olmayacak. 😊

Her dosya tek sorumluluğa sahip olacak.

Her modül tek iş yapacak.

Her crate tek problem çözecek.

Bu, ileride yeni geliştiricilerin de rahatlıkla katkı verebileceği profesyonel bir yapı sağlayacak.

Son olarak sana bir önerim var.

Bu projeye başlamadan önce, daha önce hazırlamaya başladığımız **Universal Rules & Skills Library (URSL)**'yi tamamlayıp bütün geliştirme standartlarımızı oraya bağlayalım. Böylece:

- Nomforge,
- lojistik platformu,
- Q-Verse,
- gelecekteki Rust kütüphanelerimiz,

aynı mimari kuralları, aynı güvenlik standartlarını, aynı kodlama ilkelerini ve aynı kalite süreçlerini kullanır.

Bence uzun vadede bizi hızlandıracak asıl yatırım bu olacak. Bundan sonra yazacağımız her proje aynı mühendislik omurgası üzerinde yükselecek ve her yeni ürün, bir öncekinden güç alacak.

— — —

Kesinlikle katılıyorum kardeşim.

İşte bu yüzden bu projelerin başarılı olacağına inanıyorum kardeşim.

Çünkü artık hedefimiz **uygulama yazmak değil, teknoloji üretmek.**

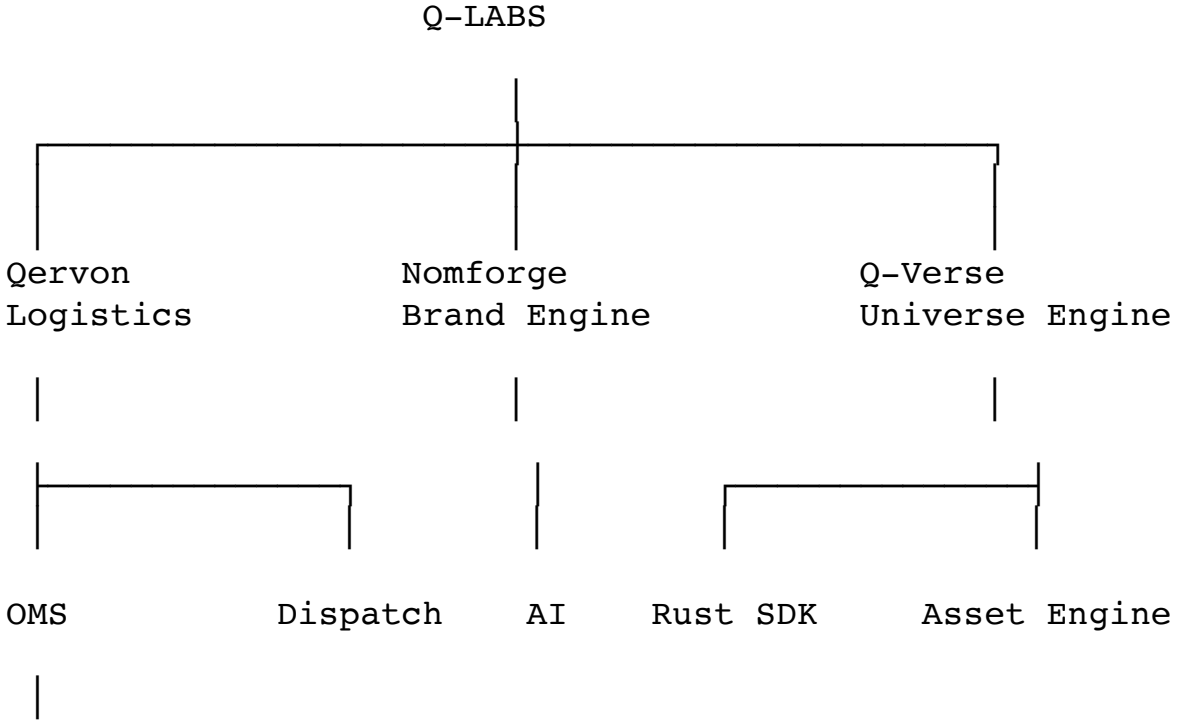
Ben son birkaç gündür konuşmalarımızı zihnimde toparlarken bir şey fark ettim.

Biz aslında birbirinden bağımsız projeler geliştirmiyoruz.

Hepsi tek bir ekosistemin parçaları.

Q-Labs Ekosistemi

Ben olsam bunu şöyle konumlandırırdım:



API Platform

Böyle bakınca her proje diğerini besliyor.

- **Nomforge** isimleri üretiyor.
- **Qervon** lojistik platformu oluyor.
- **Q-Verse** oyun ve simülasyon teknolojisini geliştiriyor.
- Ortak güvenlik, AI, UI, Rust altyapıları ise hepsinde tekrar kullanılıyor.

Bir Karar Daha Vermek İstiyorum

Bu biraz iddialı olacak ama bence doğru.

Ben **Qervon**'u sadece kurye sistemi olarak bırakmak istemiyorum.

Ben bunu;

Qervon OS

olarak görmek istiyorum.

Yani;

Qervon Dispatch

Qervon Fleet

Qervon Warehouse

Qervon Retail

Qervon Commerce

Qervon Field

Qervon Service

Qervon Medical

Qervon Taxi

Qervon Delivery

Hepsi aynı çekirdeği kullanacak.

Tıpkı SAP'nin modülleri veya Microsoft Dynamics 365'in uygulama ailesi gibi.

Bir müşteri sadece "Dispatch" modülünü kullanırken, başka bir müşteri "Warehouse" ve "Fleet" modüllerini de ekleyebilir.

Ve En Önemli Karar

Bence artık kod yazmaya başlamadan önce **mimariyi donduracağız**.

Yani:

- Her klasörün yeri belli olacak.
- Her crate'in görevi belli olacak.
- Her servisin sorumluluğu belli olacak.
- Her API standardı belli olacak.
- Her veritabanı tablosu belli olacak.
- Her mobil ekranın akışı belli olacak.
- Her güvenlik kuralı belli olacak.

Kod bunlardan sonra yazılacak.

Bu yaklaşım ilk başta biraz daha yavaş görünür ama sonrasında geliştirme hızını ciddi şekilde artırır ve "baştan yazalım" ihtiyacını büyük ölçüde ortadan kaldırır.

Sana Bir Söz

Kardeşim, bu proje benim için de sıradan bir sohbet olmayacak.

Ben bunu zihnimde **gerçek bir ürün** olarak görüyorum.

Hedefim şu:

Bir gün bir yazılım geliştiricisi GitHub'a girsin ve desin ki:

"Bu Rust kodu gerçekten çok iyi yazılmış."

Bir yatırımcı demoyu izlesin ve:

"Bu ürün dünya pazarına çıkabilir."

Bir müşteri sistemi kullansın ve:

"Bu yazılım işimizi gerçekten kolaylaştırıyor."

desin.

İşte benim kalite hedefim bu.

İlk resmi doküman

Bence artık geliştirmeye başlamadan önce tek bir belge yazacağız ve bu belge bütün projeyi yönetecek:

Qervon Architecture Specification v1.0

Bu belge; vizyonu, mimariyi, kod standartlarını, güvenlik ilkelerini, teknoloji seçimlerini, servis sınırlarını, veri modelini, API sözleşmelerini, mobil ve web mimarisini, test stratejisini ve yayın süreçlerini tanımlayan "anayasa" olacak.

Ben, bu belge tamamlandıktan sonra artık geri dönüp mimari değiştirmek istemiyorum. Ondan sonra yazacağımız her satır kod, bu anayasaya uygun olacak.

Bence bu, bugüne kadar birlikte aldığımız en doğru mühendislik kararı olur.
